VIDEORAG • TECHNICAL HANDOVER

Technical Stack & Module Reference

A source-derived guide to architecture, ingestion, retrieval, APIs, data stores, frontend modules, evaluation, fallbacks, and operational constraints.

CODE-DERIVED | FULL MODULE INVENTORY | PDF HANDOVER

Repository	MogSquad-FinalYear/VideoRAG
Snapshot	`98ae3f3eec185e84ae1d3d912cdcbb42db978d1a`
Report date	25 October 2026
Deliverable	VideoRAG_Technical_Stack_and_Modules.pdf

VideoRAG for Courtroom Evidence

Cross-session speaker identification • testimony contradiction detection • citation-verified video question answering

Contents

Contents 2

1. Executive summary 4

2. System context and architecture 4

2.1 Simplified runtime and data-boundary view . . . 4
2.2 Major runtime blocks . . . 5
2.3 Trust and data boundary . . . 5
2.4 Concurrency model . . . 6

3. Complete technology stack 6

3.1 Application platform . . . 6
3.2 AI and machine learning . . . 6
3.3 Video, data, and persistence . . . 7

4. End-to-end ingestion pipeline 7

4.1 Upload and task creation . . . 7
4.2 Processing stages . . . 7
4.3 Frame and timestamp policy . . . 7
4.4 Caption modes . . . 8

5. End-to-end query pipeline 8

5.1 Stage 1 — query decoupling . . . 8
5.2 Stage 2 — routed retrieval . . . 8
5.3 Stage 3 — merge and answer . . . 9
5.4 Stage 4 — citations and clips . . . 9

6. Agent tool catalogue 9

7. Backend module inventory 9

7.1 Core modules . . . 9
7.2 API routers . . . 10
7.3 Agent and façade . . . 10
7.4 Service modules . . . 11

8. Frontend module inventory 11

8.1 Entry and integration . . . 11
8.2 UI components . . . 12

9. API reference 13

10. Data contracts 13

11. Persistence model 14

11.1 Filesystem . . . 14

11.2 ChromaDB collections . . . 14

11.3 SQLite databases and tables . . . 14

12. Configuration reference 15

13. Evaluation suite and recorded results 15

13.1 Evaluation modules . . . 16

13.2 Interpretation limits . . . 16

14. Tests and research assets 16

15. Runtime and deployment procedure 16

15.1 Backend . . . 16

15.2 Frontend . . . 17

16. Reliability and fallback matrix 17

17. Security, privacy, and evidentiary risks 17

18. Known gaps and recommended next engineering steps 17

19. Repository map 18

20. Glossary 19

1. Executive summary

VideoRAG is a research-grade, full-stack retrieval-augmented generation system for searching and reviewing video evidence. The application accepts evidence videos, derives multiple synchronized representations of their content, and answers questions with timestamped sources and playable clips.

The design combines two forms of retrieval:

- **Semantic retrieval:** OpenCLIP embeddings stored in ChromaDB for frames, captions, speech transcripts, and OCR text.
- **Structured retrieval:** SQLite queries over objects, scene relationships, speaker identities, testimony, contradictions, and citation audit records.

The base system is extended by three courtroom-oriented novelty layers:

1. Cross-session speaker matching through diarization, Resemblyzer voiceprints, a case registry, and human confirmation.
2. Case-level testimony memory with cross-session contradiction detection.
3. Citation verification with evidence re-checking and tamper-evident frame hashes.

Media processing and data stores run locally. Groq is the external reasoning provider for query decomposition, final answer synthesis, statement-pair classification, and citation verification. The system is therefore local-first, but not fully offline when these features are enabled.

2. System context and architecture

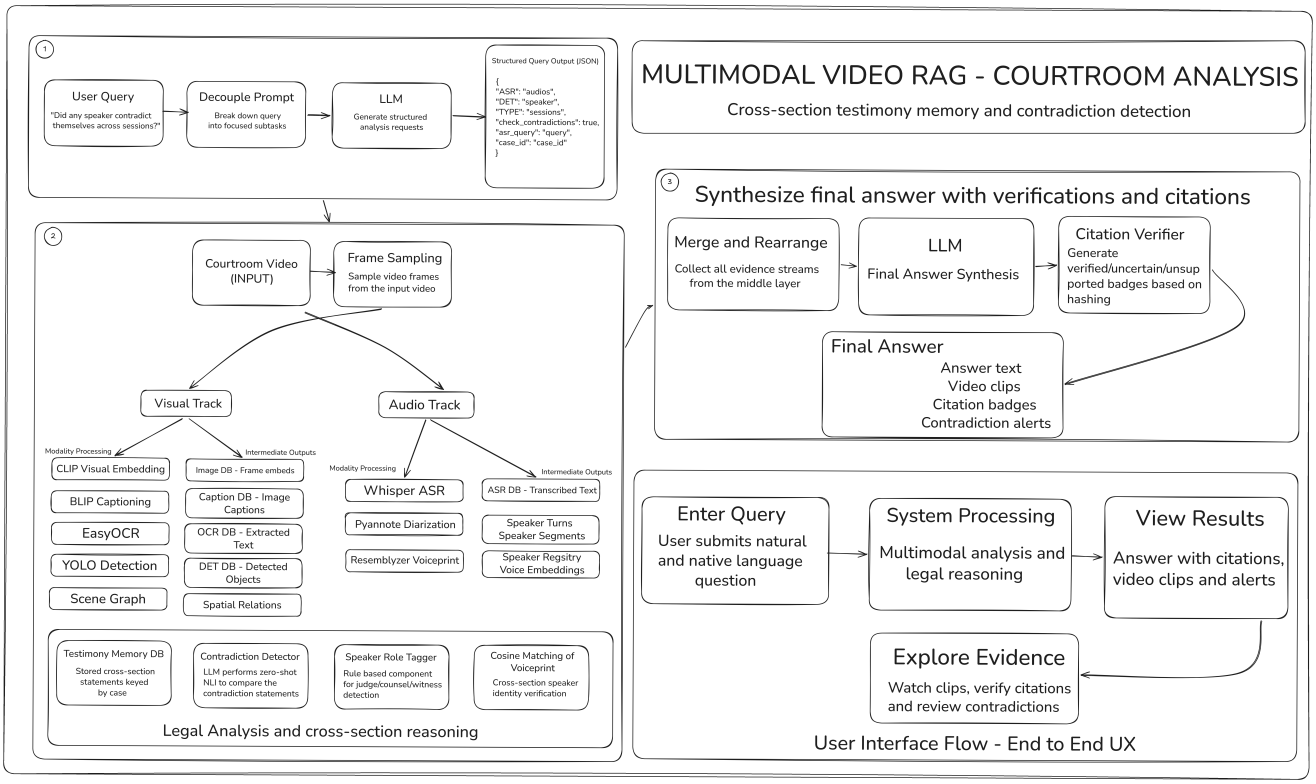


Figure. Detailed VideoRAG courtroom-analysis architecture and end-to-end workflow.

2.1 Simplified runtime and data-boundary view

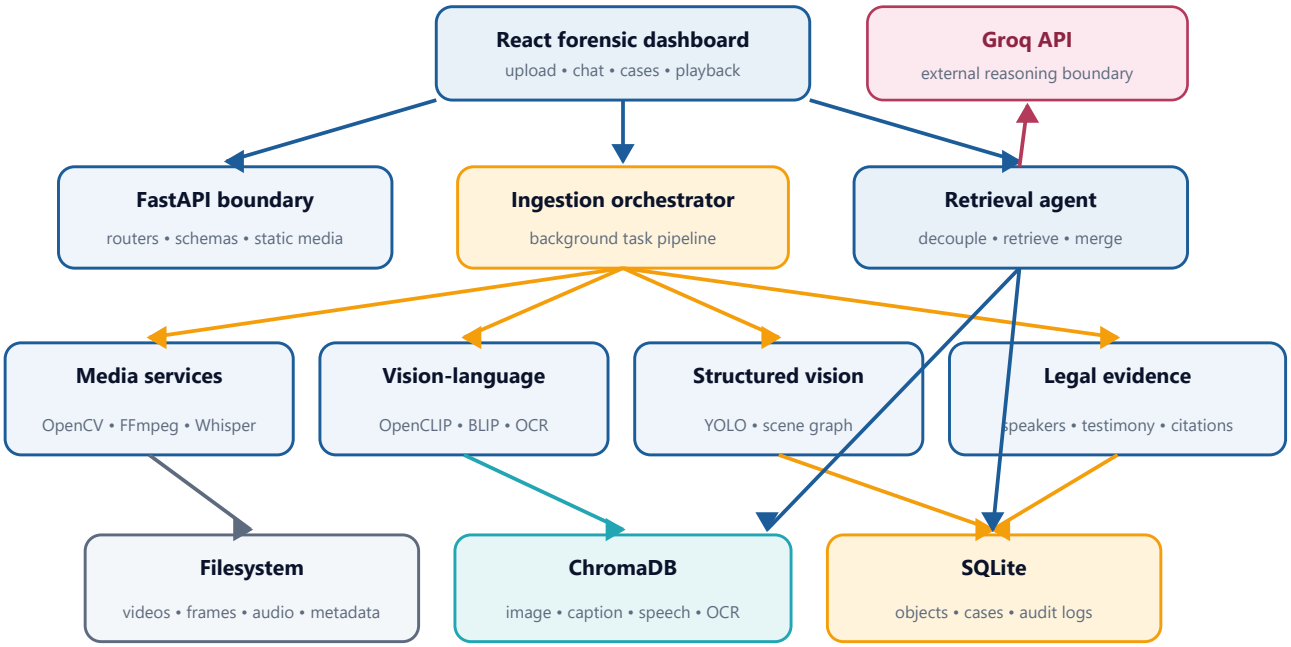


Figure 1. Source-derived runtime architecture and data boundary.

2.2 Major runtime blocks

Block	Role	Primary implementation
Forensic dashboard	Upload, evidence selection, chat, playback, case review	React/Vite in `frontend/src`
HTTP/API boundary	Validation, routing, task polling, static evidence serving	FastAPI in `backend/main.py` and `backend/routers`
Ingestion orchestrator	Runs the complete video-to-index pipeline in a background thread	`backend/routers/execute.py`
Local ML pipeline	Frames, audio, ASR, embeddings, captions, OCR, objects, speakers	`backend/services`
Retrieval agent	Decomposes questions, routes retrieval, merges evidence, drafts answers	`backend/agent/agent.py`
Vector persistence	Similarity search for image, caption, speech, and OCR evidence	ChromaDB in `data/chromadb`
Structured persistence	Exact queries, relationships, cross-session memory, audit trails	SQLite databases in `data`
External reasoning	Structured query plans, synthesis, contradiction and citation decisions	Groq API
Evaluation and paper	Reproducible metrics, figures, and research manuscript	`evaluation` and `paper`

2.3 Trust and data boundary

Local components handle uploaded files, frame and audio extraction, model inference, embeddings, vector storage, SQLite storage, and media serving. When a Groq key is configured, selected query and evidence text is sent to Groq for the four reasoning stages described above. No authentication or authorization layer is implemented around the checked-in HTTP endpoints.

2.4 Concurrency model

Uploads are streamed to disk and each video is processed in a Python daemon thread. Task state is a process-local dictionary. Asynchronous caption mode and contradiction scanning also use daemon threads. This is suitable for a single research process, but it is not a durable job queue and does not coordinate multiple backend workers.

3. Complete technology stack

3.1 Application platform

Area	Technology/version in source	Function
Backend language	Python 3.10+	API, ML, retrieval, persistence, evaluation
Web framework	FastAPI 0.115.6	REST endpoints and automatic OpenAPI docs
ASGI server	Uvicorn 0.34.0 with standard extras	Local/backend process
Data contracts	Pydantic via FastAPI	Typed request and response schemas
Multipart/config	python-multipart 0.0.20, python-dotenv 1.0.1	Uploads and `.env` loading
Frontend language	JavaScript/JSX	Browser client
UI framework	React 19.2.4, React DOM 19.2.4	Component and state model
Build/dev server	Vite 8.0.1 range; lockfile resolves 8.0.3	HMR, proxy, production build
Linting	ESLint 9.39.4 plus React plugins	Frontend static analysis
Styling	Plain CSS	Theme tokens, layouts, components, animations

The Vite 8 package lock declares Node.js `^20.19.0 || >=22.12.0`.

3.2 AI and machine learning

Capability	Technology	Checked-in default/use
Reasoning LLM	Groq client 0.13.0+	`llama-3.3-70b-versatile`, configurable
Speech recognition	OpenAI Whisper from GitHub	`tiny`, configurable
Vision-language embeddings	OpenCLIP 2.29.0+	`ViT-B-32`, LAION-2B weights
Image captioning	Hugging Face Transformers 4.47.1+	`Salesforce/blip-image-captioning-base`
Fast caption alternative	OpenCLIP label similarity	Enabled with `CAPTION_BACKEND=clip`
OCR	EasyOCR 1.7.0+	English text, CPU reader
Object detection	Ultralytics 8.3.0+	`yolov8n`, configurable
Speaker diarization	pyannote.audio 3.1+	`pyannote/speaker-diarization-3.1`
Speaker embeddings	Resemblyzer 0.1.3+	256-dimensional voiceprints
Runtime	PyTorch 2.5.1+, TorchVision 0.20.1+	GPU when available for supported models
Image support	Pillow 11+, ImageHash 4.3.1+	Model input and image utilities
Numerical support	NumPy 1.26.4+	Embeddings and model calculations

3.3 Video, data, and persistence

Capability	Technology	Purpose
Video/frame processing	OpenCV headless 4.10+	Metadata, frame sampling, scene-change scoring
Audio extraction	imageio 2.36+ and imageio-ffmpeg 0.5.1+	Bundled FFmpeg executable and WAV output
Vector database	ChromaDB 0.5.23+ persistent client	Four multimodal collections
Structured database	Python `sqlite3`	Detection, testimony, speaker, and citation tables
Static evidence serving	FastAPI `StaticFiles`	Extracted frames and uploaded videos
Evaluation	NumPy, Matplotlib, scikit-learn	Metrics, ROC/PR curves, confusion matrices, figures
Research paper	LaTeX	`paper/paper.tex` and generated figures

Matplotlib and scikit-learn are imported by evaluation scripts but are not currently pinned in `backend/requirements.txt`.

4. End-to-end ingestion pipeline

4.1 Upload and task creation

`POST /execute` accepts a multipart video and optional case ID. Supported extensions are MP4, AVI, MOV, MKV, WebM, FLV, and WMV. The browser enforces a 500 MiB limit; the backend validates extension but does not independently enforce that size limit. Files are written in 1 MiB chunks. The API returns a short task ID, video ID, status, and case ID immediately.

If no case ID is supplied, the backend creates `case_<video_id>`. This means all videos have a case scope, but cross-session behavior only emerges when several uploads reuse the same case ID.

4.2 Processing stages

Stage	Operation	Output	Failure behavior
1	Read duration, resolution, FPS, and frame count	Metadata JSON	Fatal if the outer pipeline raises
2	Adaptive scene-change frame sampling	JPEG frames and real timestamps	Falls back to uniform sampling
3	Extract audio with bundled FFmpeg	WAV file	No-audio path skips ASR
4	Batch OpenCLIP frame embedding	Image collection records	Fatal if core embedding/indexing fails
5	Caption selected frames	Caption collection records	Logged and non-fatal; sync/async/off modes
6	EasyOCR every third valid frame	OCR collection records	Logged and non-fatal
7	YOLO every second valid frame	Detection rows	Logged and non-fatal
8	Derive rules-based spatial triples	Scene-graph rows	Part of non-fatal detection block
9	Whisper transcription	Timestamped segments	Skipped when audio is absent
10	Diarize and assign speaker/role	Tagged ASR segments	pyannote falls back to time gaps
11	Voiceprint matching and registration	Speaker registry and video mappings	Logged and non-fatal
12	Index transcript segments	Speech collection records	Follows transcription
13	Store case testimony	Testimony rows	Logged and non-fatal
14	Scan prior testimony	Contradiction rows	Background, logged and non-fatal

4.3 Frame and timestamp policy

Adaptive extraction uses OpenCV frame positions and video FPS to retain real timestamps. The number of saved frames is capped by `MAX_FRAMES_PER_VIDEO` (default 300). This avoids the old assumption that a saved-frame number was equivalent to seconds elapsed, which caused large drift on long recordings.

4.4 Caption modes

- `sync`: generate/index captions before the task completes.
- `async`: complete the primary task while a daemon thread finishes captions; queries detect that captions are pending and avoid partially populated search.
- `off`: disable caption generation.

The caption backend is either BLIP or CLIP-label matching. Detailed descriptions of top returned frames use stored captions first, then on-demand BLIP, then a CLIP-label description, and finally timestamp metadata.

5. End-to-end query pipeline

5.1 Stage 1 — query decoupling

The agent asks Groq to return JSON containing:

- `asr_query`
- `visual_query`
- `caption_query`
- `ocr_query`
- `det_classes`
- `need_relations`
- `check_contradictions`
- `answer_type`

The answer type is one of count, location, description, timestamp, relation, contradiction, or general. If the call or JSON parsing fails, the agent uses a local default plan. Additional keyword rules force transcript and contradiction searches for relevant courtroom questions.

5.2 Stage 2 — routed retrieval

Strategy	Backing operation	Returned evidence
Speech/transcript	CLIP text embedding against speech collection	Segment, speaker, role, start/end, score
Visual similarity	CLIP text embedding against image collection	Frame, timestamp, similarity score
Caption search	Semantic plus literal keyword search	Frame caption and timestamp
OCR search	CLIP text embedding against OCR collection	Visible text and frame timestamp
Reference-image search	CLIP image embedding against image collection	Similar frames
Object count	SQLite grouped detection query	Maximum per-frame count and frame coverage
Object location	SQLite detection query	Boxes, centers, timestamps, confidence
Scene relations	SQLite scene-graph query	Subject–relation–object triples
Contradiction lookup	SQLite case query	Conflicting statements, sessions, confidence

Results are deduplicated by video, frame/timestamp, and segment boundaries and sorted by similarity. The reference-image path uses a 0.35 minimum score and a separate 0.30 hard gate before reporting that a match exists.

5.3 Stage 3 — merge and answer

The agent builds one evidence context containing frame descriptions, transcript segments, OCR text, detections, locations, relations, and contradictions. Groq is instructed to use only supplied evidence, use `[MM:SS]` timestamps, respect speaker/role labels, and explain contradictions using both sessions. If the LLM is unavailable or all attempts fail, a deterministic renderer produces a structured evidence response.

5.4 Stage 4 — citations and clips

Sources are converted into playable two-second windows and nearby results are merged. For ordinary text queries scoped to one video, timestamps in the final answer are parsed and checked against speech, captions, OCR, and nearby frame content. Each verification receives a result (`verified`, `uncertain`, or `unsupported`), confidence, explanation, and SHA-256 hash derived from nearby frame bytes.

6. Agent tool catalogue

Tool schema	Purpose	Current implementation path
`search_by_caption`	Natural-language scene-caption retrieval	Chroma caption semantic + keyword search
`search_by_visual_similarity`	Text-to-frame retrieval	OpenCLIP text embedding + image collection
`search_transcripts`	Dialogue/testimony retrieval	Speech collection
`search_by_image`	Reference-image-to-video retrieval	OpenCLIP image embedding + image collection
`get_video_clip_info`	Frames in a time interval	Chroma metadata with filesystem fallback
`search_ocr`	On-screen text retrieval	OCR collection
`count_objects`	Per-class maximum object count	Detection SQLite database
`locate_object`	Object boxes and positions	Detection SQLite database
`get_relations`	Spatial relationship triples	Scene-graph SQLite table
`check_contradictions`	Case/speaker contradiction records	Testimony SQLite database

`backend/agent/tools.py` contains Groq-compatible JSON function schemas. The current `agent.py` path primarily uses structured decoupling followed by direct dispatch through `_execute_tool`; it does not import the schema catalogue.

`backend/mcp_server/server.py` exposes overlapping helpers through an `MCPTools` class. It is an in-process façade, not a standalone MCP transport or network listener.

7. Backend module inventory

7.1 Core modules

File	Responsibilities and notable behavior
`backend/__init__.py`	Package marker
`backend/main.py`	Creates FastAPI 1.0.0 app; configures logging and CORS; mounts frames/videos; registers routers; reports root/health state
`backend/config.py`	Finds `.env`; creates data directories; defines model names, sampling/caption settings, ports, collection names, and database paths
`backend/models.py`	Pydantic schemas for upload/status, chat/search/clips, citation/contradiction results, video metadata, roles, and speaker confirmation

7.2 API routers

File	Responsibilities and notable behavior
routers/__init__.py	Router package marker
routers/execute.py	Upload streaming, task dictionary, full processing orchestrator, caption scheduling, and cascading delete across every store
routers/status.py	Returns task state or HTTP 404
routers/chat.py	Text/reference-image endpoints; validates Groq availability for chat; saves temporary query images; maps agent output into Pydantic models
routers/videos.py	Reads metadata JSON into evidence-library records and redirects playback to static video files
routers/speakers.py	Reads/writes roles; returns pending/canonical matches; confirms or renames voiceprint matches
routers/testimony.py	Returns case testimony and contradictions with optional speaker/confidence filters

7.3 Agent and façade

File	Responsibilities and notable behavior
agent/__init__.py	Agent package marker
agent/agent.py	Query expansion, Groq client, source enrichment, structured decoupling, tool dispatch, multimodal collection, evidence merge, fallback answers, clip creation, citation handoff
agent/tools.py	Ten function schemas covering retrieval and legal tools
mcp_server/__init__.py	MCP-style package marker
mcp_server/server.py	Seven static helper methods and singleton `mcp_tools` wrapper

7.4 Service modules

File	Responsibilities and notable behavior
`services/__init__.py`	Service package marker
`video_processor.py`	Locates imageio FFmpeg; reads metadata; uniform extraction; adaptive scene-change extraction with spacing/cap constraints
`embedding_service.py`	Singleton OpenCLIP model/preprocess/tokenizer; chooses CUDA when available; normalized image/text embeddings; validated batch output
`captioning_service.py`	Singleton BLIP; fixed CLIP label vocabulary; batched captions; detailed multi-prompt descriptions; CLIP fallback
`transcription_service.py`	Makes bundled FFmpeg discoverable to Whisper; singleton Whisper model; CUDA/CPU selection; normalizes segments to `start_time`/`end_time`
`indexing_service.py`	Singleton persistent Chroma client and four collections; path normalization; indexing; similarity/keyword search; stats/readiness; delete and point lookup
`ocr_service.py`	Singleton EasyOCR English reader; confidence/length filtering; bounding boxes; strided batch extraction
`detection_service.py`	Singleton YOLO model; confidence-filtered boxes; strided batch detection; helper counts and spatial labels
`detection_db.py`	Detection and relation schema initialization; batch inserts; class/time filters; count, locate, relation, and delete queries
`scene_graph_service.py`	IoU, center, and distance helpers; produces inside, overlap, on-top-of, left/right, above/below, and near triples
`speaker_service.py`	pyannote availability/model handling; overlap alignment; gap fallback; role cues; Resemblyzer voiceprints; cosine registry match; human confirmation; cleanup
`testimony_db.py`	Testimony and contradiction schemas; case/speaker/video retrieval; batch statement storage; contradiction storage and cleanup
`contradiction_service.py`	Groq three-way classifier; JSON validation; scans new segments against prior statements; persists only confident contradictions
`citation_service.py`	Parses `[MM:SS]` timestamps; gathers nearby evidence; hashes frame bytes; asks Groq for support decisions; writes audit records
`citation_db.py`	Citation-log schema, inserts, and answer/video retrieval

8. Frontend module inventory

8.1 Entry and integration

File	Responsibility
`frontend/index.html`	Vite HTML host
`frontend/src/main.jsx`	Mounts React under strict mode
`frontend/src/App.jsx`	Owns selected video/case, upload visibility, refresh counter, active clip, sidebar state, and case panel state
`frontend/src/api/client.js`	Eleven async API operations plus task polling; XHR upload used when progress reporting is requested
`frontend/vite.config.js`	React plugin, port 5173, and path-specific proxy to `127.0.0.1:8000`
`frontend/eslint.config.js`	ESLint rules for JavaScript and React hooks/refresh

8.2 UI components

Component	Responsibility
`Sidebar.jsx`	Loads evidence library, selects video/case scope, opens upload/case UI, deletes videos, shows thumbnails/status/duration
`UploadModal.jsx`	Drag/drop and browse flow, extension/MIME validation, 500 MiB client limit, optional case ID, upload/progress polling, pending-match discovery
`ChatWindow.jsx`	Text and image query composer, chat history, loading/errors, selected scope, starter prompts, and clip callback
`MessageBubble.jsx`	Safe lightweight Markdown-like rendering and structured cards for clips, sources, citation verification, and contradictions
`VideoPlayer.jsx`	Video URL, clip seeking, duration/timestamp formatting, close action, and extracted-frame gallery
`CasePanel.jsx`	Speakers/Testimony/Contradictions tabs; confirms or renames matches; edits roles; displays statements and paired conflicts
`StatusBadge.jsx`	Processing, ready, failed, and missing state display

Every component has a colocated CSS file. `index.css` defines global theme and base styles; `App.css` defines the top-level three-panel layout and overlays.

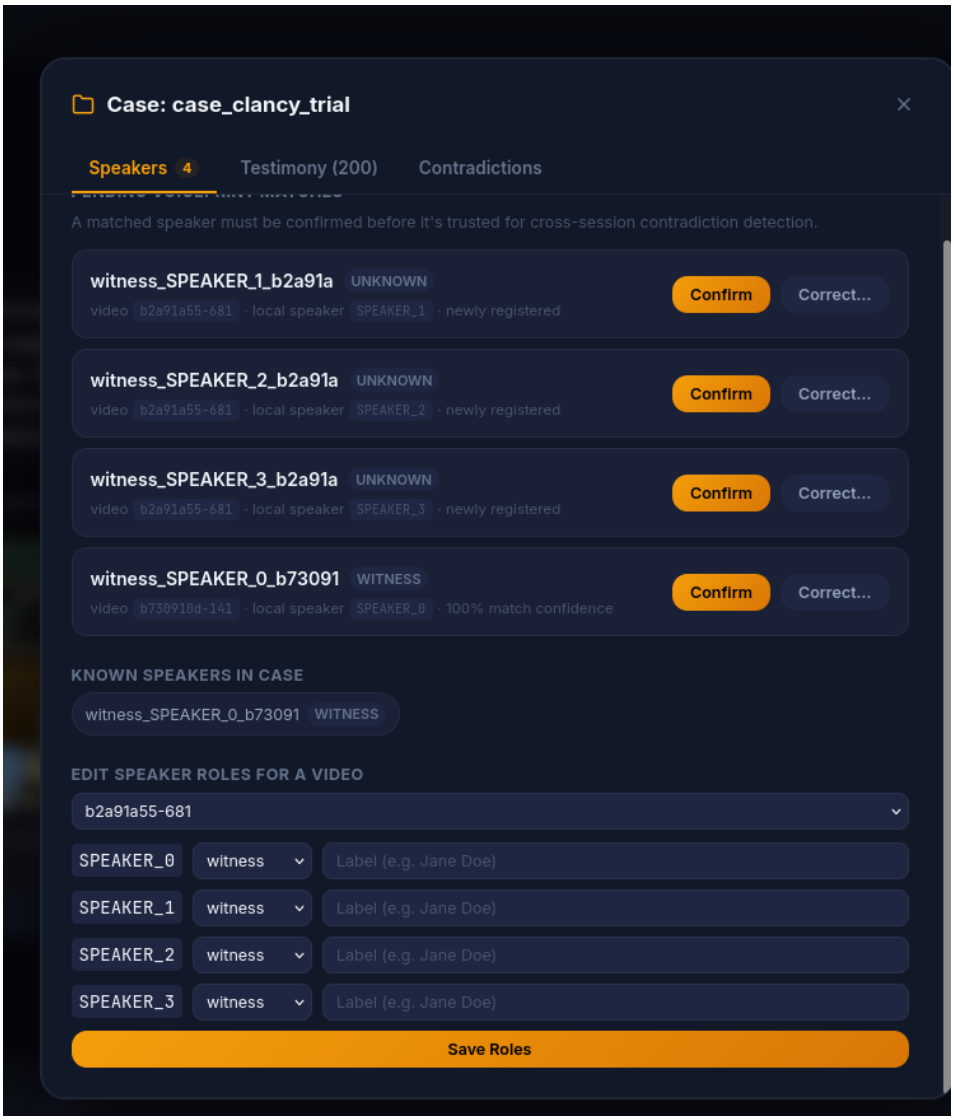


Figure 2. Case panel for speaker confirmation and role management.

9. API reference

Method	Route	Input	Output/purpose
GET	`/`	None	Basic API status and `/docs` pointer
GET	`/health`	None	Chroma counts plus detection/testimony/citation database availability
POST	`/execute`	Multipart `file`, optional `case_id`	Starts processing and returns task/case IDs
GET	`/status/{task_id}`	Path task ID	Status, progress, message, video ID, caption readiness
GET	`/videos`	None	All metadata-backed evidence records
GET	`/videos/{video_id}/play`	Path video ID	Redirect to `/video-files/<stored-file>`
DELETE	`/videos/{video_id}`	Path video ID	Cascading media, index, metadata, detection, testimony, and speaker cleanup
POST	`/chat`	JSON query, optional video/case	Answer, clips, sources, citations, contradictions
POST	`/chat/image`	Multipart query, image, optional video	Image-to-video answer, clips, and sources
GET	`/videos/{video_id}/speakers`	Path video ID	Role/label mappings
POST	`/videos/{video_id}/speakers`	Mapping list	Upserts manual roles/labels
GET	`/cases/{case_id}/speaker-matches`	Path case ID	Pending mappings plus registered canonical speakers
POST	`/cases/{case_id}/speaker-matches/confirm`	Match action payload	Confirms or renames mapping
GET	`/cases/{case_id}/testimony`	Optional limit	Ordered case statements
GET	`/cases/{case_id}/contradictions`	Optional speaker and confidence	Ordered contradiction records

Static paths `/frames/...` and `/video-files/...` expose derived frames and uploaded evidence. The development UI accesses the API through Vite proxy rules.

10. Data contracts

Pydantic model	Key fields
`ExecuteResponse`	`task_id`, `status`, `message`, optional `case_id`
`TaskStatus`	`task_id`, `status`, `progress`, `message`, optional `video_id`, `captions_ready`
`ChatRequest`	`query`, optional `video_id`, `image_path`, `case_id`
`SearchResult`	Video/frame/timestamps, score, content, frame path, source index
`VideoClip`	Video ID, start/end, frame paths, description
`CitationVerification`	ID, timestamp, score, result, explanation, clip hash
`ContradictionResult`	IDs, two statements/videos/timestamps, confidence, explanation
`ChatResponse`	Answer plus clip, source, citation, and contradiction arrays
`VideoInfo`	Identity, file metadata, status, thumbnail, caption readiness, case ID
`SpeakerRoleMapping`	Local speaker ID, role, human label
`SpeakerRoleMappingRequest`	List of mappings
`SpeakerMatch`	Video/local speaker, canonical name, confidence, role, confirmation state
`SpeakerMatchConfirmRequest`	Video/local speaker, confirm-or-rename action, optional corrected name/role

11. Persistence model

11.1 Filesystem

Path	Content
`data/videos`	Original uploaded evidence under generated video IDs
`data/frames/<video_id>`	Sampled JPEG frames
`data/audio/<video_id>.wav`	Extracted audio
`data/metadata/<video_id>.json`	Source filename, media properties, status, thumbnail, caption state, case ID
`data/chromadb`	Persistent vector collections
`data/detections.db`	Structured vision data
`data/testimony.db`	Speaker and testimony data
`data/citations.db`	Citation verification log

11.2 ChromaDB collections

Default collection	Documents/embeddings	Important metadata
`image_index`	OpenCLIP frame image embeddings	Video, frame, timestamp, browser frame path
`caption_index_clip`	CLIP text embeddings of captions	Video, frame, timestamp, path
`speech_index_clip`	CLIP text embeddings of ASR text	Video, segment times, speaker, role
`ocr_index`	CLIP text embeddings of visible text	Video, frame, timestamp, path

11.3 SQLite databases and tables

Database	Table	Purpose
`detections.db`	`detections`	One object class, box, confidence, video/frame/timestamp per row
`detections.db`	`scene_graph`	Subject–relation–object triples tied to frames/timestamps
`testimony.db`	`speaker_roles`	Manual/automatic local speaker roles and labels
`testimony.db`	`speaker_registry`	Case-wide canonical speakers and representative voiceprints
`testimony.db`	`speaker_video_map`	Local-to-canonical mappings, confidence, confirmation, candidate voiceprint
`testimony.db`	`testimony_memory`	Case/session/video/speaker/role/timestamp statement ledger
`testimony.db`	`contradictions`	Statement pairs, source sessions, confidence, and explanation
`citations.db`	`citation_log`	Claim, timestamp/video, score, decision, SHA-256 hash, verification time

Deletion removes Chroma records, detection/relation rows, uploaded media, frames, audio, metadata, testimony, and speaker mappings. Citation rows are not currently included in the cascading video delete path.

12. Configuration reference

Variable	Default	Meaning
`GROQ_API_KEY`	Empty	Enables external LLM reasoning
`HF_TOKEN`	Empty	Passed to the gated pyannote pipeline loader
`GROQ_MODEL`	`llama-3.3-70b-versatile`	Groq model identifier
`WHISPER_MODEL`	`tiny`	Whisper model size/name
`CLIP_MODEL`	`ViT-B-32`	OpenCLIP architecture
`CLIP_PRETRAINED`	`laion2b_s34b_b79k`	OpenCLIP weights
`BLIP_MODEL`	`Salesforce/blip-image-captioning-base`	Caption model
`YOLO_MODEL`	`yolov8n`	Ultralytics model basename
`FRAME_SAMPLE_FPS`	`1`	Uniform extraction fallback rate
`MAX_FRAMES_PER_VIDEO`	`300`	Frame processing cap
`CAPTION_STRIDE`	`3`	Caption candidate stride
`CAPTION_MAX_FRAMES`	`24`	Maximum captioned frames per video
`CAPTION_MODE`	`sync`	`sync`, `async`, or `off`
`CAPTION_BACKEND`	`blip`	`blip` or `clip`
`ENABLE_LLM_POLISH`	`false`	Parsed feature flag; not currently referenced elsewhere
`BACKEND_PORT`	`8000`	Configuration value; startup command remains explicit
`FRONTEND_PORT`	`5173`	Configuration value; Vite file remains explicit
`IMAGE_COLLECTION`	`image_index`	Chroma image collection name
`CAPTION_COLLECTION`	`caption_index_clip`	Chroma caption collection name
`SPEECH_COLLECTION`	`speech_index_clip`	Chroma speech collection name
`OCR_COLLECTION`	`ocr_index`	Chroma OCR collection name

The environment loader checks repository `.env`, `backend/.env`, and the current working directory. Existing variables are overwritten by a discovered file because `load_dotenv(..., override=True)` is used.

13. Evaluation suite and recorded results

The evaluation directory contains the methodology, seven scripts, JSON output, standalone figures, and paper-ready copies. These are exploratory measurements from the repository's development corpus, not a large external benchmark.

Evaluation	Recorded result	Sample
Timestamp fidelity	Maximum reconstructed pre-fix drift corrected: 1,094.39 s on a long video	Seven videos; two 22+ minute courtroom videos
Speech retrieval	Exact-frame Recall@10 0.575; exact MRR 0.472; content-aware MRR 0.566	40 queries, 697 documents
Caption retrieval	Exact-frame Recall@10 0.150; exact MRR 0.021	40 queries, 81 documents
OCR retrieval	Exact-frame Recall@10 0.050; exact MRR 0.011; content-aware MRR 0.000	40 queries, 228 documents
Reference-image search	ROC-AUC 0.959, EER 0.021 at threshold 0.842	73 positive and 616 negative frames
Speaker verification	ROC-AUC 1.0; precision/recall 1.0 at 0.75	10 pairs: 3 same, 7 different
Contradiction detection	Accuracy 1.0; contradiction false-positive rate 0.0	21 designed/end-to-end pairs

Evaluation	Recorded result	Sample
Citation verification	Accuracy 1.0 vs. 0.5 trust-all baseline	10 balanced cases
Scalability	22-minute videos processed in roughly 143–151 s in recorded runs	15 runs

13.1 Evaluation modules

File	Function
`evaluation/EVALUATION.md`	Corpus, methods, results, limitations, and suggested follow-up
`scripts/eval_common.py`	Dataset registry, paper mode, accessible palette, plot style, JSON/figure output
`scripts/eval_01_timestamp_fidelity.py`	Monotonic/bounded timestamp checks and reconstructed old-formula drift
`scripts/eval_02_retrieval.py`	Self-query Recall@k and MRR, exact-frame and content-aware
`scripts/eval_03_image_search.py`	Cosine scores, ROC/PR curves, EER, deployed operating points
`scripts/eval_04_speaker_verification.py`	Pairwise voiceprint similarities and configured-threshold confusion
`scripts/eval_05_contradiction_detection.py`	Three-class precision/recall/F1 and false-positive rate
`scripts/eval_06_citation_verification.py`	Binary support classification versus naive baseline
`scripts/eval_07_scalability.py`	Wall-clock duration, warm/cold labels, voiceprint memory-fix comparison
`results/*.json`	Raw metrics and detailed records
`figures/*.png`	Standalone evaluation figures
`paper/figures/*.png`	Title-free paper figures

13.2 Interpretation limits

The perfect scores for speaker verification, contradiction detection, and citation verification come from small, clean evaluation sets. They demonstrate that the implemented path works on those cases; they do not establish courtroom deployment accuracy. Caption and OCR retrieval results expose genuine weaknesses in repetitive captions and using CLIP text embeddings for short literal OCR strings. The image-search evaluation also shows that the checked-in thresholds produce many false positives despite a strong ROC-AUC.

14. Tests and research assets

The repository root contains `test_debug.py`, `test_direct.py`, `test_legal_features.py`, `test_queries.py`, `test_strategies.py`, and `test_verify.py`. These are direct/integration/debug scripts rather than a unified pytest suite; there is no checked-in CI workflow or coverage threshold. `test_result.json`, `test_output.txt`, and the screenshots/images are captured development artifacts.

`paper/paper.tex` is the accompanying ICMLDE manuscript and consumes figures from `paper/figures`. Several implementation-plan Markdown files preserve the project's design evolution but are not imported at runtime.

`tools/generate_tech_stack_pdf.py` is the reproducible ReportLab renderer for this Markdown source. It provides the cover, table of contents, vector architecture diagram, table styling, screenshot placement, headers, footers, and PDF metadata.

15. Runtime and deployment procedure

15.1 Backend

1. Create and activate a Python virtual environment.
2. Install `backend/requirements.txt`.
3. Add `.env` with `GROQ_API_KEY` and optional `HF_TOKEN`/overrides.

4. Run `python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000`.
5. Verify `/health` and `/docs`.

First use may download Whisper, OpenCLIP, BLIP, YOLO, EasyOCR, pyannote, and Resemblyzer model assets. This creates a much larger cold-start time and requires network access and sufficient storage.

15.2 Frontend

1. Use Node.js `^20.19.0` or `>=22.12.0`.
2. Run `npm install` in `frontend`.
3. Run `npm run dev` and open `http://localhost:5173`.
4. For release checking, run `npm run lint` and `npm run build`.

The Vite proxy maps `/execute`, `/status`, `/chat`, `/videos`, `/video-files`, `/frames`, `/health`, and `/cases` to the local backend. There is no generic `/api` prefix in the checked-in configuration.

16. Reliability and fallback matrix

Feature	Preferred path	Fallback/degradation
Frame sampling	Adaptive scene-change sampler	Uniform sampler
Captioning	Configured BLIP or CLIP batch backend	Failure logged; other indexes remain usable
Top-frame description	Stored caption then BLIP	CLIP-label description then timestamp-only text
Diarization	pyannote speaker turns	Silence-gap speaker changes
Voice identity	Resemblyzer embedding and registry	Disabled when package/model fails
Query planning	Groq JSON decoupling	Local multimodal default plus keywords
Answer synthesis	Groq evidence-grounded response	Deterministic evidence renderer
OCR/detection	EasyOCR/YOLO	Non-fatal skip
Contradiction scan	Background Groq classifier	Non-fatal skip/no new flags
Citation verification	Evidence retrieval plus Groq decision	Non-fatal; answer still returned

17. Security, privacy, and evidentiary risks

- There is no login, role-based access control, case isolation, or per-route authorization.
- CORS permits the two local Vite origins, but this is not an authentication boundary.
- Media and frames are available through static URLs to anyone who can reach the backend and knows or discovers the path.
- Data is stored unencrypted in local files, ChromaDB, and SQLite.
- There is no malware scan, MIME sniffing, quota, backend upload-size limit, or retention/secure-erasure policy.
- Groq prompts may contain evidence text, statements, and generated context.
- SHA-256 citation hashes cover nearby frame bytes, but the project does not implement a signed chain of custody, trusted timestamps, or immutable audit storage.
- Model outputs and similarity scores are probabilistic. They require human review and must not be represented as independent legal findings.
- Background work and task state are not durable across restart.
- SQLite writes and local ChromaDB access are not designed here for high concurrency or distributed deployment.

18. Known gaps and recommended next engineering steps

1. Replace in-memory task state and daemon threads with a durable queue and persisted jobs.
2. Add authentication, authorization, case-level tenancy, encrypted storage, upload quotas, and evidence-retention controls.
3. Pin evaluation dependencies and establish a real automated test/CI suite.
4. Calibrate image similarity thresholds against representative evidence.
5. Replace CLIP OCR embeddings with a dedicated text embedding or hybrid lexical retrieval path.
6. Improve caption diversity and benchmark the BLIP/CLIP modes separately.
7. Add real multi-session speaker benchmarks, diarization error rate, and human-annotated contradiction/citation audits.
8. Make configured ports drive Uvicorn/Vite launch behavior consistently.
9. Either expose a standards-compliant MCP transport or rename the in-process façade to avoid implying a deployed MCP server.
10. Include citation-log cleanup in video deletion or define an explicit immutable-retention policy.

19. Repository map

```
VideoRAG/
├── backend/
│   ├── agent/                query planning, tool schemas, synthesis
│   ├── mcp_server/           in-process search façade
│   ├── routers/              FastAPI endpoints
│   ├── services/             media/ML/retrieval/legal/persistence modules
│   ├── config.py
│   ├── main.py
│   ├── models.py
│   └── requirements.txt
├── frontend/
│   ├── public/               favicon and SVG sprite
│   ├── src/api/client.js     HTTP client
│   ├── src/components/       dashboard components and CSS
│   ├── src/App.jsx           top-level application
│   ├── src/main.jsx          browser entry
│   └── package.json
├── evaluation/
│   ├── scripts/              seven evaluations plus common helpers
│   ├── results/              raw JSON metrics
│   ├── figures/              standalone figures
│   └── EVALUATION.md
├── paper/                    LaTeX manuscript and paper figures
├── docs/                     this report in Markdown and PDF
├── tools/                    technical-report PDF generator
├── test_*.py                 direct/integration/debug scripts
├── README.md
└── LICENSE
```

20. Glossary

Term	Meaning in this project
ASR	Automatic speech recognition performed by Whisper
RAG	Retrieval-augmented generation: retrieve evidence before producing an answer
CLIP	Shared image/text embedding model used for cross-modal similarity
BLIP	Image captioning model used for frame descriptions
OCR	Optical character recognition for visible text
Diarization	Assignment of speech intervals to speaker identities
Voiceprint	Numeric speaker embedding used for cross-session matching
Scene graph	Structured subject–relation–object description of frame objects
Case	User-supplied grouping of videos that share speakers and testimony memory
Session	One uploaded video within a case
Citation verification	Re-checking an answer's timestamped claim against nearby indexed evidence

---

This report documents the code as checked out at the commit shown on the cover. It distinguishes implemented behavior from planned or implied capabilities and should be regenerated after material code, dependency, API, or data-schema changes.